
Trabajo Grupal - Primer Parcial

Física Computacional

Nombres de los Integrantes:

Mamani Huarsaya, Jorge Luis

Aragón Carpio Fredy José

Ayamamani Anasco, Francis Willian

Choquehuanca Bedoya Brayan Denilson

Estudiante 5

13 de mayo de 2026

Índice

1. Movimiento parabólico	3
1.1. Enunciado	3
1.2. Aplicación de la ecuación	3
1.3. Aplicación del método de Heun	5
1.4. Comparación de resultados	7
2. Movimiento de 2 cuerpos	8
2.1. Enunciado	8
2.2. Modelo físico	8
2.3. Normalización de unidades	9
2.4. Implementación numérica	9
2.5. Resultados	10
2.6. Tipo de trayectoria	11
2.7. Conclusiones	11
3. Simulación de 5 cuerpos gravitacionales	12
3.1. Modelo y aceleraciones	12
3.2. Implementación	12
4. Movimiento oscilatorio	15
4.1. Modelo	15
4.2. Casos y configuración	16
4.3. Implementación numérica	16

1. Movimiento parabólico

1.1. Enunciado

La trayectoria de una partícula es (extraído del libro de Serway o Sears):

$$y = x \tan \alpha_0 - \frac{g}{2v_0^2 \cos^2 \alpha_0} x^2 \quad (1)$$

donde v_0 y α_0 son las condiciones iniciales.

1. Mediante una gráfica determine el alcance mediante la ecuación de arriba. Las condiciones iniciales son $v_0 = 5 \text{ m/s}$ con $\alpha_0 = 60^\circ$.
2. Aplicar el método de Heun y encuentre el alcance.
3. Haga la comparación de los resultados encontrados. Para ello varíe el tamaño de paso h para que el resultado sea confiable.

1.2. Aplicación de la ecuación

Para resolver la primera parte del problema, se utilizó directamente la ecuación analítica del movimiento parabólico mostrada en la Ecuación 1. Se implementó un programa en Python que genera múltiples valores de la posición horizontal x y evalúa la altura correspondiente $y(x)$ para cada punto.

Las condiciones iniciales consideradas fueron $v_0 = 5 \text{ m/s}$, $\alpha_0 = 60^\circ$ y $g = 10 \text{ m/s}^2$. Luego, se graficó la trayectoria de la partícula y se observó la forma parabólica característica del movimiento y determinar visualmente el alcance horizontal.

La implementación utilizada se muestra en el Código 1.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parámetros
5 g = 10
6 v0 = 5
7 alpha = np.radians(60)
8
9 # Ecuación
10 x = np.linspace(0, 2.3, 500)
11 y = x * np.tan(alpha) - (g / (2 * v0**2 * np.cos(alpha)**2)) * x**2
12
13 # Encontrar el alcance
14 indice = np.where(y >= 0)[0][-1]
15 alcance = x[indice]
16
17 # Gráfica
18 plt.plot(x, y)
19 plt.scatter(alcance, 0)
20 plt.text(alcance, 0, f'{alcance:.4f}')
21 plt.grid(True)
22 plt.xlabel('x (m)')
23 plt.ylabel('y (m)')
24 plt.title('Movimiento parabólico - Analítico')
25
26 plt.savefig('../images/problem01_1.png', dpi=300, bbox_inches='tight')
27 plt.show()
```

Código 1: Implementación de la gráfica de la ecuación 1.

La Figura 1 muestra la trayectoria obtenida mediante la ecuación analítica.

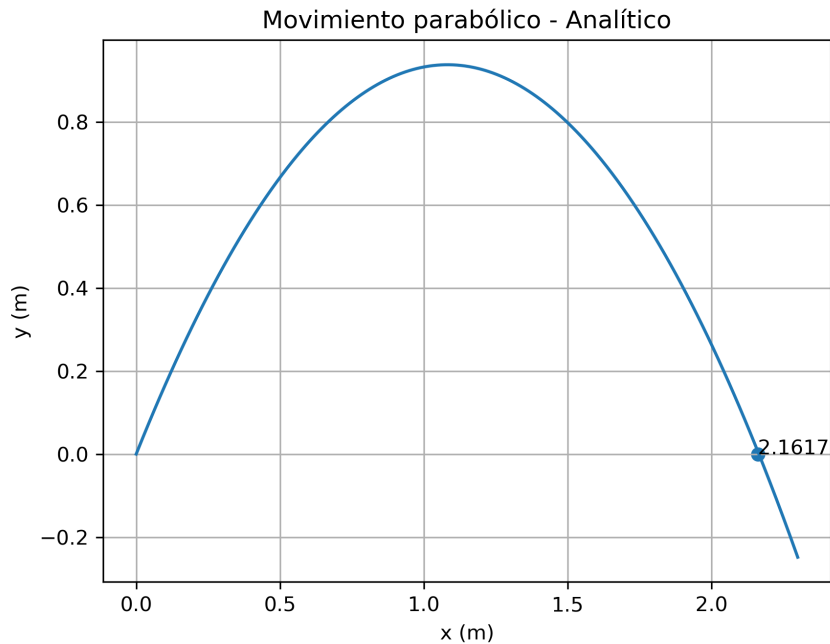


Figura 1: Solución analítica.

A partir de la gráfica obtenida, se observa que el proyectil alcanza aproximadamente una dis-

tancia horizontal de 2.16 m antes de regresar al suelo.

1.3. Aplicación del método de Heun

Para resolver numéricamente el movimiento parabólico, se empleó el método de Heun, el cual corresponde a un método predictor-corrector de segundo orden utilizado para aproximar soluciones de ecuaciones diferenciales ordinarias.

El movimiento de la partícula puede describirse mediante el siguiente sistema:

$$\frac{dx}{dt} = v_x \quad (2)$$

$$\frac{dy}{dt} = v_y \quad (3)$$

$$\frac{dv_x}{dt} = 0 \quad (4)$$

$$\frac{dv_y}{dt} = a \quad (5)$$

donde la aceleración gravitacional se considera constante:

$$a = -10 \text{ m/s}^2 \quad (6)$$

Las componentes iniciales de la velocidad se calcularon a partir de la velocidad inicial y el ángulo de lanzamiento:

$$v_{x0} = v_0 \cos \alpha_0 \quad (7)$$

$$v_{y0} = v_0 \sin \alpha_0 \quad (8)$$

El método de Heun se aplicó utilizando un esquema predictor-corrector. En cada iteración, primero se estimó una velocidad vertical predictora mediante el método de Euler y posteriormente se corrigió la posición vertical utilizando el promedio entre la pendiente inicial y la pendiente predicha.

La integración numérica se realizó para intervalos de tiempo separados por un tamaño de paso $h = 0.01$. La simulación continuó hasta que la partícula regresó al suelo, es decir, cuando la coordenada vertical tomó valores menores que cero.

La implementación desarrollada se muestra en el Código 2.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parámetros
5 a = -10
6 v0 = 5
7 alpha = np.radians(60)
8
9 vx = v0 * np.cos(alpha)
10 vy = v0 * np.sin(alpha)
11
12 x = 0
13 y = 0
14
15 h = 0.01
16 tfin = 50
17
18 px = [x]
19 py = [y]
20
21 for t in np.arange(0, tfin, h):
22
23     vy_pred = vy + h * a
24
25     y = y + h * (vy + vy_pred) / 2
26     x = x + h * vx
27
28     vy = vy_pred
29
30     px.append(x)
31     py.append(y)
32
33     if y < 0:
34         break
35
36 # Último punto físico válido
37 alcance = px[-2]
38
39 # Gráfica
40 plt.plot(px, py)
41 plt.scatter(alcance, 0)
42 plt.text(alcance, 0, f'{alcance:.4f}')
43 plt.grid(True)
44 plt.xlabel('x (m)')
45 plt.ylabel('y (m)')
46 plt.title('Movimiento parabólico - Método de Heun')
47
48 plt.savefig('../images/problem01_2.png', dpi=300, bbox_inches='tight')
49 plt.show()
```

Código 2: Implementación del método de Heun para el movimiento parabólico.

La Figura 2 muestra la trayectoria obtenida mediante el método numérico.

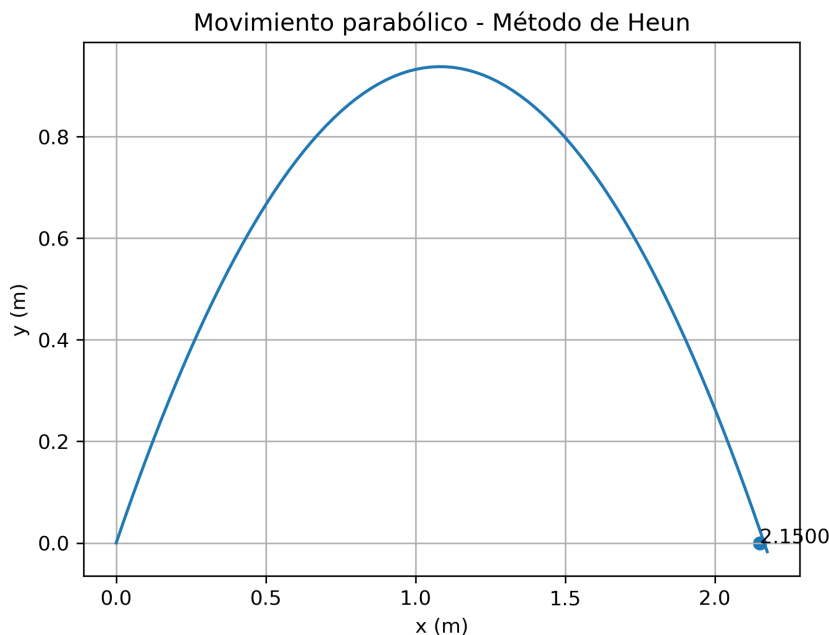


Figura 2: Trayectoria obtenida mediante el método de Heun.

El alcance horizontal obtenido mediante el método de Heun fue aproximadamente 2.16 m, mostrando una buena concordancia con el resultado analítico obtenido previamente.

1.4. Comparación de resultados

Finalmente, se realizaron simulaciones utilizando distintos tamaños de paso h con el objetivo de analizar la convergencia del método de Heun. Para cada simulación, el alcance horizontal se determinó tomando el último punto válido antes de que la trayectoria atravesase el suelo, es decir, antes de que la coordenada vertical tome valores negativos.

Posteriormente, los resultados numéricos fueron comparados con el alcance obtenido mediante la solución analítica desarrollada en la primera parte del problema.

Los resultados obtenidos se muestran en la Tabla 1.

Paso h	Alcance obtenido (m)	Error relativo (%)
0.1	2.0000	7.4812
0.01	2.1500	0.5423
0.001	2.1650	0.1516

Tabla 1: Comparación del alcance para diferentes tamaños de paso.

Se observa que el alcance calculado mediante el método numérico depende directamente del tamaño de paso utilizado durante la integración temporal. Para valores grandes de h , la simulación produce una aproximación menos precisa debido a que el método avanza mediante intervalos discretos relativamente amplios.

Al disminuir el tamaño de paso, los puntos calculados se aproximan progresivamente a la trayectoria analítica y el error relativo disminuye considerablemente. Esto evidencia la convergencia del método de Heun hacia la solución exacta del problema.

En conclusión, el método de Heun proporciona resultados satisfactorios para el movimiento pa-

rábólico y mejora su precisión conforme se reduce el tamaño de paso empleado en la simulación.

2. Movimiento de 2 cuerpos

2.1. Enunciado

Se estudió el movimiento del objeto interestelar *3I/ATLAS*, el cual atravesó el sistema solar siguiendo una trayectoria abierta alrededor del Sol. El objetivo fue modelar numéricamente su movimiento utilizando la ley de gravitación universal y datos aproximados obtenidos de información astronómica disponible públicamente.

Para el desarrollo del problema se consideraron:

- Masa del Sol.
- Radio del Sol.
- Velocidad aproximada de 3I/ATLAS.
- Movimiento gravitacional de dos cuerpos.

El sistema se simplificó considerando únicamente la interacción gravitacional entre el Sol y el objeto interestelar.

2.2. Modelo físico

La fuerza gravitacional entre el Sol y el objeto se modeló mediante la ley de gravitación universal:

$$\vec{F} = -\frac{GMm}{r^3}\vec{r} \quad (9)$$

donde:

- G es la constante de gravitación universal.
- M es la masa del Sol.
- m es la masa del objeto.
- r es la distancia entre ambos cuerpos.

A partir de esta expresión se obtuvieron las aceleraciones en cada eje:

$$a_x = -\frac{x}{(x^2 + y^2)^{3/2}} \quad (10)$$

$$a_y = -\frac{y}{(x^2 + y^2)^{3/2}} \quad (11)$$

Estas ecuaciones fueron implementadas numéricamente en MATLAB para calcular la trayectoria del objeto alrededor del Sol.

2.3. Normalización de unidades

Con el objetivo de simplificar los cálculos numéricos, las distancias fueron expresadas en radios solares.

Se tomó:

$$R_{\odot} = 696\,350 \text{ km} \quad (12)$$

Por tanto:

$$1 \text{ unidad} = 1R_{\odot} \quad (13)$$

Las velocidades obtenidas de información astronómica fueron inicialmente expresadas en km/h y posteriormente convertidas a radios solares por hora dividiendo entre el radio solar.

Las componentes aproximadas de velocidad utilizadas fueron:

$$v_x \approx 62\,689 \text{ km/h} \quad (14)$$

$$v_y \approx -211\,922 \text{ km/h} \quad (15)$$

Luego de la conversión:

$$v_x \approx 0.090 \quad (16)$$

$$v_y \approx -0.304 \quad (17)$$

Las posiciones iniciales también fueron expresadas en radios solares:

$$x_0 = -458.33079 \quad (18)$$

$$y_0 = 2665.95683 \quad (19)$$

Estos valores fueron estimados utilizando una hoja de cálculo para realizar conversiones y aproximaciones temporales hacia atrás desde el perihelio.

2.4. Implementación numérica

La trayectoria se calculó utilizando integración temporal mediante el método de Euler explícito.

La implementación utilizada se muestra en el Código 3.

```

1 function ax = aceleracionx(x,y,a,b)
2     ax = -((x-a)/((x-a)^2+(y-b)^2)^1.5);
3 end
4 function ay = aceleraciony(x,y,a,b)
5     ay = -((y-b)/((x-a)^2+(y-b)^2)^1.5);
6 end
7 clear, clf, hold off; n=0;
8 % Constantes del Sistema
9 %M_sol = 1.9885e30;
10 M_sol = 1; %1 masa de sol
11 %g_sol = 274;
12 g_sol = 1; %1 gravedad del sol
13 %R=0.7e9;
14 R=1; % 1 radio del sol
15 h=1; tfin=16000;
16 GM = M_sol*g_sol;
17 a = 0;
18 b = 0;
19 th2 = 0:0.01:2*pi;
20 xunit2 = a + 1 * cos(th2);
21 yunit2 = b + 1 * sin(th2);
22 hold on;
23 plot(xunit2, yunit2, 'g');
24 % Condiciones iniciales
25 x=-458.33079; y=2665.95683; vx=0.090026; vy=-0.304333; px(1)=x; py(1)=y; n=0;
26 for t=0:h:tfin
27     ax=(aceleracionx(x,y,a,b)*GM)/R^2; ay=(aceleraciony(x,y,a,b)*GM)/R^2;
28     vx = vx + ax*h; x = x + vx*h;
29     vy = vy + ay*h; y = y + vy*h;
30     px(n+1) = x; py(n+1) = y; n=n+1;
31 end
32 plot(px,py);
33 grid on;
34 xlim([-3000 3000]);
35 ylim([-3000 3000]);

```

Código 3: Simulación numérica de la trayectoria de 3I/ATLAS.

La simulación se realizó utilizando un tamaño de paso pequeño para reducir el error numérico durante la integración.

2.5. Resultados

La Figura 3 muestra la trayectoria obtenida para el objeto interestelar al pasar cerca del Sol.

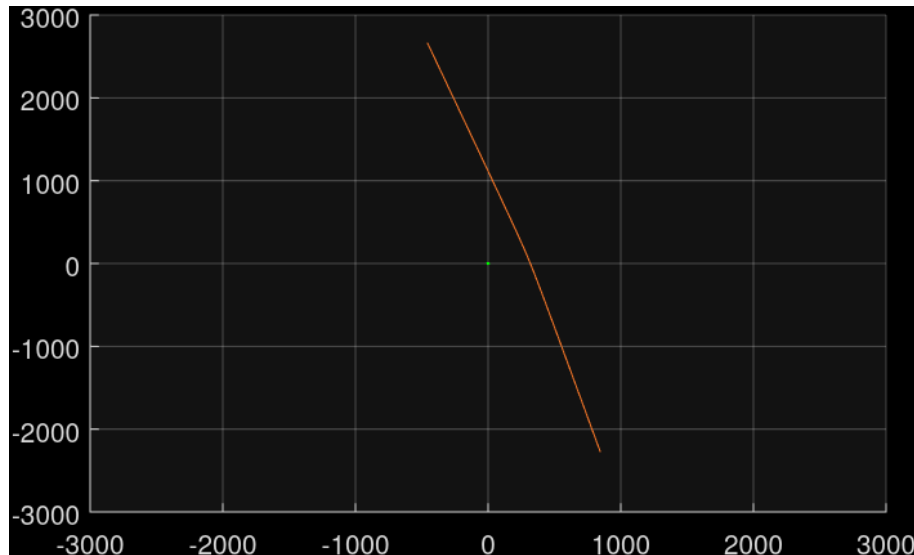


Figura 3: Trayectoria aproximada de 3I/ATLAS alrededor del Sol.

En la figura se observa una trayectoria abierta característica de un objeto interestelar que no permanece ligado gravitacionalmente al Sol.

Debido a pequeñas variaciones decimales en las condiciones iniciales y en las conversiones realizadas, la trayectoria puede desplazarse aproximadamente entre 10 y 20 radios solares. Sin embargo, el comportamiento general permanece consistente y conserva la forma hiperbólica esperada.

Asimismo, la gráfica presenta una trayectoria visualmente muy abierta. Esto ocurre porque el objeto posee una velocidad extremadamente alta y la gravedad solar únicamente desvía parcialmente su movimiento.

2.6. Tipo de trayectoria

La trayectoria obtenida corresponde a una órbita hiperbólica.

Una órbita hiperbólica ocurre cuando la energía mecánica total del sistema es positiva, es decir, cuando el objeto posee suficiente velocidad para escapar permanentemente de la atracción gravitacional del Sol.

La excentricidad orbital cumple:

$$e > 1 \quad (20)$$

En el caso de 3I/ATLAS, los datos astronómicos indican una excentricidad aproximada:

$$e \approx 6.3 \quad (21)$$

lo cual confirma que el objeto proviene del espacio interestelar y continuará alejándose del sistema solar después de su acercamiento al Sol.

2.7. Conclusiones

La simulación permitió modelar de manera aproximada el movimiento de un objeto interestelar utilizando el problema gravitacional de dos cuerpos.

Los resultados obtenidos muestran una trayectoria hiperbólica abierta, coherente con las observaciones astronómicas reportadas para 3I/ATLAS.

Además, se verificó que pequeñas variaciones numéricas en las condiciones iniciales producen desplazamientos apreciables en la trayectoria, lo cual evidencia la sensibilidad del sistema a errores de redondeo y aproximación durante las conversiones numéricas.

3. Simulación de 5 cuerpos gravitacionales

Simular el movimiento de 5 cuerpos gravitacionales. Los primeros 4 se ubican en los vértices de un cuadrado de lado a . Encontrar las aceleraciones, modificar las funciones de dichas aceleraciones y graficar las trayectorias resultantes.

3.1. Modelo y aceleraciones

El sistema consiste en cinco cuerpos que interactúan entre sí mediante la ley de gravitación. A diferencia de un sistema con un único centro de fuerza, cada cuerpo experimenta la atracción simultánea de los otros cuatro, lo que genera trayectorias altamente no lineales e impredecibles a simple vista.

La aceleración del cuerpo i producida por la influencia gravitacional de los demás cuerpos se expresa como:

$$\vec{a}_i = \sum_{j \neq i} \frac{Gm_j(\vec{r}_j - \vec{r}_i)}{(|\vec{r}_{ij}|^2 + \varepsilon^2)^{3/2}} \quad (22)$$

donde $\varepsilon = 0.15$ es un parámetro de suavizado que evita la divergencia numérica cuando dos cuerpos se aproximan excesivamente. Sin este término, la magnitud de la aceleración tendería a infinito al reducirse la distancia entre cuerpos, desestabilizando completamente la integración numérica.

Los cuatro cuerpos exteriores de masa $m = 1$ se ubican en los vértices de un cuadrado de lado $a = 2$. El cuerpo central, con masa $m_5 = 5$, ejerce una atracción cinco veces mayor que cada uno de los cuerpos exteriores. Esto implica que los cuatro cuerpos en las esquinas sienten una fuerza dominante hacia el centro, mientras que las interacciones entre ellos generan perturbaciones laterales que desvían sus trayectorias del movimiento circular ideal. El resultado es un sistema caótico donde pequeñas variaciones en las condiciones iniciales o en el tamaño de paso producen trayectorias cualitativamente distintas.

3.2. Implementación

La implementación se organiza en tres bloques. El primero define los parámetros físicos del sistema: la constante gravitacional $G = 1$, las masas, el lado del cuadrado $a = 2$, el parámetro de suavizado ε y el paso de integración $h = 0.002$ con tiempo final $t_{\text{fin}} = 15$. Luego se establecen las posiciones iniciales de los cinco cuerpos y sus velocidades tangenciales, calculadas a partir de la velocidad orbital $v_{\text{orb}} = 0.80\sqrt{GM_{\text{total}}/R}$ con $R = a/\sqrt{2}$, de modo que el sistema parta con cierta inercia rotacional. Finalmente se inicializan los arreglos que almacenarán las trayectorias a lo largo de la simulación.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

```

3  # Parámetros del sistema
4  G      = 1.0
5  m      = 1.0
6  m5     = 5.0
7  a      = 2.0
8  eps    = 0.15
9  h      = 0.002
10 tfin   = 15

```

```

27 # Condiciones iniciales
28 # C1:(+a/2,+a/2) C2:(-a/2,+a/2)
29 # C3:(-a/2,-a/2) C4:(+a/2,-a/2) C5:(0,0)
30 masas = np.array([m, m, m, m, m5])
31 N      = 5
32
33 rx = np.array([ a/2, -a/2, -a/2,  a/2,  0.0])
34 ry = np.array([ a/2,  a/2, -a/2, -a/2,  0.0])
35
36 R      = a / np.sqrt(2)
37 v_orb  = 0.80 * np.sqrt(G * np.sum(masas) / R)
38
39 vx = np.array([-v_orb,  v_orb,  v_orb, -v_orb,  0.0])
40 vy = np.array([ v_orb,  v_orb, -v_orb, -v_orb,  0.0])
41
42 # Inicializar arreglos
43 nmax = int(round(tfin / h)) + 2
44 prx  = np.zeros((N, nmax))
45 pry  = np.zeros((N, nmax))
46
47 n = 0
48 prx[:, 0] = rx
49 pry[:, 0] = ry

```

Código 4: Parámetros del sistema, condiciones iniciales e inicialización de arreglos.

El segundo bloque contiene la función **aceleraciones**, que recibe las posiciones actuales de todos los cuerpos y calcula la aceleración neta sobre cada uno aplicando la ecuación gravitacional con suavizado. Seguido del bucle de integración mediante el método de Euler, donde en cada paso se evalúan las aceleraciones, se actualizan las velocidades y posiciones, y se registran las coordenadas en los arreglos de trayectoria.

```

12 # Función de aceleraciones
13 def aceleraciones(rx, ry, masas, G, eps):
14     N = len(masas)
15     ax = np.zeros(N)
16     ay = np.zeros(N)
17     for i in range(N):
18         for j in range(N):
19             if i != j:
20                 dx = rx[j] - rx[i]
21                 dy = ry[j] - ry[i]
22                 r3 = (dx**2 + dy**2 + eps**2)**1.5
23                 ax[i] = ax[i] + G * masas[j] * dx / r3
24                 ay[i] = ay[i] + G * masas[j] * dy / r3
25     return ax, ay

```

```

51 # Método de Euler
52 for t in np.arange(0, tfin - h, h):

```

```

53     n += 1
54     ax, ay = aceleraciones(rx, ry, masas, G, eps)
55     vx = vx + ax * h
56     vy = vy + ay * h
57     rx = rx + vx * h
58     ry = ry + vy * h
59     prx[:, n] = rx
60     pry[:, n] = ry
61
62 prx = prx[:, :n+1]
63 pry = pry[:, :n+1]

```

Código 5: Función de aceleraciones gravitacionales y método de Euler.

El tercer bloque genera la figura de trayectorias con fondo oscuro. Cada cuerpo se representa con un color distinto: la trayectoria completa se traza con baja opacidad como estela, y el último 25 % del recorrido se resalta con mayor brillo para mostrar la dirección de movimiento reciente. Los marcadores circulares indican la posición inicial y las estrellas la posición final de cada cuerpo.

```

65 # FIGURA
66 colores = [
67     [0.2, 0.5, 1.0],
68     [1.0, 0.3, 0.3],
69     [0.2, 0.9, 0.4],
70     [1.0, 0.7, 0.1],
71     [0.8, 0.3, 1.0]
72 ]
73 nombres = ['C1 (+,+)', 'C2 (-,+)', 'C3 (-,-)', 'C4 (+,-)', 'C5 centro']
74
75 fig, ax1 = plt.subplots(figsize=(12, 10))
76 fig.patch.set_facecolor([0.05, 0.05, 0.08])
77 ax1.set_facecolor([0.05, 0.05, 0.08])
78 ax1.tick_params(colors=[0.5, 0.5, 0.5])
79 ax1.xaxis.label.set_color([0.75, 0.75, 0.75])
80 ax1.yaxis.label.set_color([0.75, 0.75, 0.75])
81 for spine in ax1.spines.values():
82     spine.set_edgecolor([0.25, 0.25, 0.25])
83 ax1.grid(True, color=[0.25, 0.25, 0.25], alpha=0.5)
84 ax1.set_aspect('equal')
85
86 for i in range(N):
87     c = colores[i]
88     # Estela tenue
89     ax1.plot(prx[i, :], pry[i, :], '-',
90             color=c + [0.4], linewidth=1.0)
91     # Cola brillante: último 25%
92     k0 = int(round(0.75 * n))
93     ax1.plot(prx[i, k0:], pry[i, k0:], '-',
94             color=c + [1.0], linewidth=2.0, label=nombres[i])
95
96 for i in range(N):
97     ax1.plot(prx[i, 0], pry[i, 0], 'o',
98             markersize=9, markerfacecolor=colores[i],
99             markeredgecolor='k', markeredgewidth=1.2)
100
101 for i in range(N):
102     ax1.plot(prx[i, -1], pry[i, -1], 'p',
103             markersize=12, markerfacecolor=colores[i],
104             markeredgecolor='k', markeredgewidth=1.0)
105
106 ax1.set_xlabel('x')

```

```

107 ax1.set_ylabel('y')
108 ax1.set_title('Trayectorias - 5 Cuerpos Gravitacionales',
109             color='w', fontsize=14)
110 ax1.legend(loc='upper right')
111
112 plt.tight_layout()
113 plt.savefig('images/problem03.png', dpi=150,
114         facecolor=fig.get_facecolor(), bbox_inches='tight')
115 plt.show()

```

Código 6: Configuración visual y generación de la figura de trayectorias.

La Figura 4 muestra las trayectorias obtenidas para los cinco cuerpos durante el tiempo de simulación $t_{\text{fin}} = 15$.

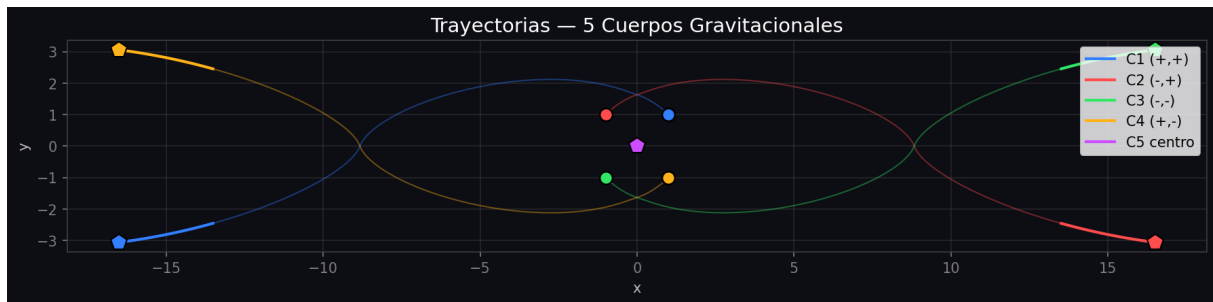


Figura 4: Trayectorias de los 5 cuerpos gravitacionales obtenidas mediante el método de Euler. Los círculos indican posiciones iniciales y las estrellas las posiciones finales.

Se observa que los cuatro cuerpos exteriores no describen órbitas cerradas sino trayectorias abiertas y asimétricas. La atracción del cuerpo central domina el movimiento global, actuando como un núcleo gravitacional que curva continuamente las trayectorias de los cuerpos periféricos. Sin embargo, las interacciones mutuas entre estos últimos generan aceleraciones transversales que rompen la simetría inicial del sistema, produciendo el comportamiento caótico visible en la figura. Los cuerpos que parten de posiciones diagonalmente opuestas tienden a alejarse en direcciones contrarias, evidenciando que el sistema no conserva la simetría rotacional más allá de los primeros instantes de la simulación.

4. Movimiento oscilatorio

Fabriqué las figuras de Lissajous en tres dimensiones extendiendo el modelo de osciladores armónicos independientes a tres coordenadas. Cada coordenada (x, y, z) obedece una ecuación de movimiento con su propia frecuencia, lo que permite generar curvas 3D al combinar razones enteras entre frecuencias.

4.1. Modelo

El sistema se representa como tres osciladores armónicos desacoplados:

$$\ddot{x} = -\frac{k_1}{m_1}x, \quad \ddot{y} = -\frac{k_2}{m_2}y, \quad \ddot{z} = -\frac{k_3}{m_3}z. \quad (23)$$

Con $m_1 = m_2 = m_3 = 1$ y $k_1 = l^2$, $k_2 = n^2$, $k_3 = p^2$, las frecuencias quedan $\omega_x = l$, $\omega_y = n$, $\omega_z = p$. Las condiciones iniciales se fijan en $x_0 = y_0 = z_0 = 1$, $v_{x0} = 2.36$, $v_{y0} = v_{z0} = 0$.

4.2. Casos y configuración

Se evalúan siete ternas (l, n, p) para generar familias distintas de curvas 3D:

$$\{(1, 2, 1), (1, 3, 1), (2, 3, 1), (3, 4, 1), (3, 5, 2), (4, 5, 3), (5, 6, 4)\}. \quad (24)$$

El tiempo de integración es $t_{\text{fin}} = 50$ con paso $h = 0.01$. Para cada terna se almacena la trayectoria completa y se grafica en un mosaico 3×3 con `plot3`.

4.3. Implementación numérica

El avance temporal se realiza con el método de Euler-Cromer. Primero se actualizan las velocidades con las aceleraciones instantáneas y luego se actualizan las posiciones. Esta variante mejora la estabilidad frente al Euler simple cuando se trata de sistemas oscilatorios.

```

1  clear; clf; close all;
2
3  % Base parameters
4  m1 = 1; m2 = 1; m3 = 1; % masses (kg)
5  h = 0.01;               % integration step (s)
6  tfin = 50;              % final time (s)
7
8  % Initial conditions
9  x0 = 1; vx0 = 2.36;
10 y0 = 1; vy0 = 0.0;
11 z0 = 1; vz0 = 0.0;
12
13 % Acceleration functions
14 function ax = acel_x(x, k1, m1)
15     ax = -(k1 / m1) * x;
16 end
17
18 function ay = acel_y(y, k2, m2)
19     ay = -(k2 / m2) * y;
20 end
21
22 function az = acel_z(z, k3, m3)
23     az = -(k3 / m3) * z;
24 end
25
26 % Frequency triples (l, n, p)
27 casos = [
28     1, 2, 1; % (a)
29     1, 3, 1; % (b)
30     2, 3, 1; % (c)
31     3, 4, 1; % (d)
32     3, 5, 2; % (e)
33     4, 5, 3; % (f)
34     5, 6, 4; % (g)
35 ];
36
37 num_casos = size(casos, 1);
38
39 % Prepare figure with 3D subplots
40 figure('Name', 'Figuras de Lissajous 3D', 'NumberTitle', 'off');
41 tiledlayout(3, 3, 'TileSpacing', 'compact', 'Padding', 'compact');
42
43 for idx = 1:num_casos
44     l = casos(idx, 1);
45     n = casos(idx, 2);
46     p = casos(idx, 3);
47

```



```

48     k1 = l^2; % m1 = 1
49     k2 = n^2; % m2 = 1
50     k3 = p^2; % m3 = 1
51
52     % Storage arrays
53     N = round(tfin / h) + 1;
54     px = zeros(1, N); py = zeros(1, N); pz = zeros(1, N);
55
56     % Initial state
57     x = x0; vx = vx0;
58     y = y0; vy = vy0;
59     z = z0; vz = vz0;
60     px(1) = x; py(1) = y; pz(1) = z;
61
62     n_step = 1;
63     for t = h:h:tfin
64         % Accelerations
65         ax_val = acel_x(x, k1, m1);
66         ay_val = acel_y(y, k2, m2);
67         az_val = acel_z(z, k3, m3);
68
69         % Euler-Cromer update
70         vx = vx + ax_val * h;
71         x = x + vx * h;
72         vy = vy + ay_val * h;
73         y = y + vy * h;
74         vz = vz + az_val * h;
75         z = z + vz * h;
76
77         n_step = n_step + 1;
78         px(n_step) = x; py(n_step) = y; pz(n_step) = z;
79     end
80
81     % Trim arrays
82     px = px(1:n_step); py = py(1:n_step); pz = pz(1:n_step);
83
84     % 3D plot in a subplot
85     nexttile;
86     plot3(px, py, pz, 'b-', 'LineWidth', 0.8);
87     xlabel('x'); ylabel('y'); zlabel('z');
88     title(sprintf('l=%d, n=%d, p=%d', l, n, p));
89     grid on;
90     view(30, 30);
91     axis equal;
92 end
93
94 sgtitle('Curvas de Lissajous en tres dimensiones');

```

Código 7: Simulación de Lissajous 3D usando Euler-Cromer y visualización en subplots.

En cada subplot se usa `axis equal` para mantener proporciones reales entre ejes y `view(30,30)` para una perspectiva uniforme. El resultado es un conjunto de curvas de Lissajous tridimensionales que muestran cómo las razones enteras entre frecuencias controlan la geometría del trazo.